

作业9 调度理论

同济大学计算机系操作系统 国豪2023 拔尖班2023

姓名 毛星博 学号 2351520

一、在一个单道批处理系统中，一组作业的提交时间和运行时间如下表所示。试计算以下2种作业调度算法的平均周转时间T和平均带权周转时间W。请填表。

(1) 先来先服务； (2) 短作业优先。

作业	提交时间	运行时间
1	8.0	1.0
2	8.5	0.4
3	9.0	0.2
4	9.1	0.1

先来先服务 FIFS:

作业执行次序	提交时间	运行时间	等待时间	开始时间	完成时间	周转时间	带权周转时间
1	8.0	1.0	0	8.0	9.0	1.0	1.0
2	8.5	0.4	0.5	9.0	9.4	0.9	2.25
3	9.0	0.2	0.4	9.4	9.6	0.6	3.0
4	9.1	0.1	0.5	9.6	9.7	0.6	6.0
作业平均周转时间	$T = (1.0 + 0.9 + 0.6 + 0.6) / 4 = 0.775$						
作业平均带权周转时间	$W = (1.0 + 2.25 + 3.0 + 6.0) / 4 = 3.0625$						

短作业优先 SJF:

作业执行次序	提交时间	运行时间	等待时间	开始时间	完成时间	周转时间	带权周转时间
1	8.0	1.0	0	8.0	9.0	1.0	1.0
3	9.0	0.2	0	9.0	9.2	0.2	1.0
4	9.1	0.1	0.1	9.2	9.3	0.2	2.0
2	8.5	0.4	0.8	9.3	9.7	1.2	3.0
作业平均周转时间	$T = (1.0 + 0.2 + 0.2 + 1.2) / 4 = 0.65$						
作业平均带权周转时间	$W = (1.0 + 1.0 + 2.0 + 3.0) / 4 = 1.75$						

二、在一个多道批处理系统中，一组进程的就绪时刻和运行时间如下：

进程	就绪时刻	运行时间（剩余时间）
1	8.0	1.0
2	8.5	0.4
3	9.0	0.2
4	9.1	0.1

2.1 不可剥夺系统，分别采用 FIFS 和 短作业优先。每个进程的周转时间和平均周转时间是否与第一题一致。

答：结果与第一题的单道批处理完全一致。

因为数据相同且不可剥夺逻辑相同。这些进程没IO操作的，所有计算量一次跑完。

2.2 可剥夺系统中的短作业优先算法有一个另外的名字，叫做最短剩余时间优先。剩余时间是指就绪进程的下一CPU服务时间。比如，一个低优先级的就绪进程，原本的服务时间是10s，运行2s后被剥夺，它的剩余服务时间就是8s。另外，现运行进程也有剩余时间，语义是类似的。最后，新进程就绪，系统立即将其剩余时间与现运行进程的剩余时间相比较，若新进程剩余时间短，抢占现运行进程。

本题所示系统，最短剩余时间优先，填写下表，多出来的行自己划掉：

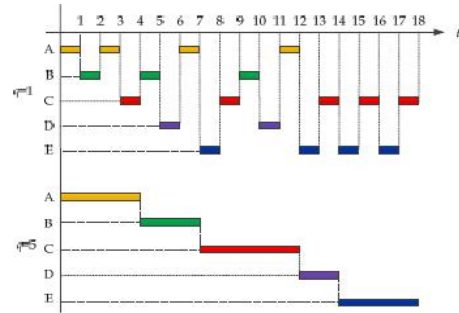
进程执行次序	就绪时刻	剩余时间	等待时间	开始时间	完成时间	周转时间	带权周转时间
1	8.0	1.0	0	8.0	8.5		
2	8.5	0.4	0	8.5	8.9	0.4	1.0
1	8.0	0.5	0.4	8.9	9.0		
3	9.0	0.2	0	9.0	9.2	0.2	1.0
4	9.1	0.1	0.1	9.2	9.3	0.2	2.0
1	8.0	0.4	0.3	9.3	9.7	1.7	1.7
进程平均周转时间		$T = (1.7 + 0.4 + 0.2 + 0.2) / 4 = 2.5 / 4 = 0.625$					
进程平均带权周转时间		$W = (1.7 + 1.0 + 1.0 + 2.0) / 4 = 1.425$					

总结：

进程	周转时间	带权周转时间
1	1.7	1.7
2	0.4	1.0
3	0.2	1.0
4	0.2	2.0

三、A、B、C、D、E 时间片轮转。运行情况如下图所示。q是时间片长度。q==5时，所有进程可以在一个时间片内运行完毕，算法蜕化成FIFS。下面我们看q=1时系统的运行情况。

进程 情况 时间 片	进程名	A	B	C	D	E	平均
	到达时间	0	1	2	3	4	
q=1	服务时间	4	3	5	2	4	
	完成时间	12	10	18	11	17	
	周转时间	12	9	16	8	13	11.6
	带权周转时间	3	3	3.2	4	3.25	3.29
q=5	完成时间	4	7	12	14	18	
	周转时间	4	6	10	11	14	9
	带权周转时间	1	2	2	5.5	3.5	2.8



时刻 就绪队列

0 A

1 BA

2 ACB

3 CBDA

4 BDAEC

5 DAECB 从时刻5开始，5个进程时间片轮转一轮。

10 DAEC 时刻10，B运行结束。现在的一轮，是4个进程轮转。在这一轮中，A、D运行结束。

14 EC E，C轮转。直至运行结束

注意（1）就绪队列首部是现运行进程，对照右图演算。（2）2s结束的时刻，C进入系统排就绪队列队尾，B用完时间片去队尾，排在C的后面。

四、单项选择题

1. 下面对进程的描述中，错误的是 D。

A. 进程是动态的概念 B. 进程执行需要处理机

C. 进程是有生命期的 D. 进程是指令的集合

进程是动态的执行过程，程序才是指令的集合

2. 多道程序环境下，操作系统分配资源以 C 为基本单位。

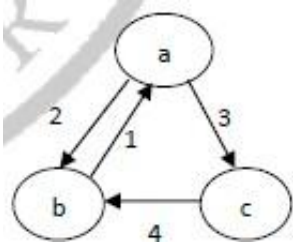
A. 程序 B. 指令 C. 进程 D. 作业

3. 操作系统通过 B 对进程进行管理。

A. 进程 B. 进程控制块 C. 进程启动程序 D. 进程控制区

4. 系统感知进程的唯一实体（进程存在的唯一标志）是 C。

A. JCB B. FCB C. PCB D. SJT

5. 在进程管理中, 当 C 时, 进程从阻塞状态变为就绪状态。
 A. 进程被进程调度程序选中 B. 进程开始等待某一事件
C. 进程等待的事件发生 D. 进程的时间片用完
6. 分配到必要的资源并获得处理机时的状态是 B。
 A. 就绪状态 **B. 执行状态** C. 阻塞状态 D. 撤销状态
7. 进程的三个基本状态之间在一定条件下是可以相互转化的, 进程由就绪状态变为运行状态的条件是: ① D; 由运行状态变为阻塞状态的条件是 ② B。
 A. 时间片用完 B. 需要等待某事件发生
 C. 等待的某事件已发生 D. 被进程调度程序选中
8. 下列的进程状态变化中, C 变化是不可能发生的,
 A. 运行→就绪 B. 运行→等待 **C. 等待→运行** D. 等待→就绪
9. 一个运行中的进程用完了分配给它的时间片后, 它的状态变为 A。
A. 就绪 B. 等待 C. 运行 D. 不确定
10. 某系统的进程状态如下图所示: a 是 ① B 状态, b 是 ② D 状态, c 是 ③ E 状态。1 表示 ④ B, 2 表示 ⑤ A, 3 表示发生了等待事件, 4 表示等待事件结束。
 ① ②③: A. 挂起 B. 运行 C. 等待 D. 就绪 E. 睡眠
 ④⑤: A. 落选 B. 选中 C. 等待
- 
11. 下面所述步骤中, A 不是创建进程所必需的。
 A. **由调度程序为进程分配CPU** B. 建立一个进程控制块
 C. 为进程分配内存 D. 将进程控制块链入就绪队列
12. 设某系统中有 n ($n > 2$) 个进程, 且当前不在执行进程调度程序, 上述情况中, 不可能发生的情况是: A
 A. **没有运行进程, 有2个就绪进程,** n 个进程处于等待状态;
 B. 有1个运行进程, 没有就绪进程, $n-1$ 进程处于等待状态;
 C. 有1个运行进程, 有1个就绪进程, $n-2$ 进程处于等待状态;
 D. 有1个运行进程, $n-1$ 个就绪进程, 没有进程处于等待状态。
只要有就绪就会调度运行, 不可能有2个就绪却没运行进程
13. 进程调度是从 A 选择一个进程投入运行。
A. 就绪队列 B. 等待队列 C. 作业后备队列 D. 提交队列
14. 一个进程在某一时刻具有 A。
A. 一种状态 B. 两种状态 C. 三种状态 D. 四种状态
15. “可抢占”和“不可抢占”的优先级调度算法相比 B。

- A. 前者开销小
B. 前者开销大
C. 两者开销大致相同
D. 两者开销不能相比

可抢占开销大，因为“可抢占”优先级调度上下文切换频繁，每调度一次都会引起保护现场、恢复现场的工作。

16. B 优先权是在创建进程时确定的，确定之后在整个进程运行期间不再改变。
A. 先来先服务 B. 静态 C. 动态 D. 短作业

17. 各种作业调度算法中，若所有作业同时到达，则平均等待时间最短的算法是 D。

- A. 先来先服务 B. 优先数 C. 最高响应比优先 D. 短作业优先
短作业优先能获得最短的平均等待/周转时间

18. A 是指从作业提交给系统到作业完成的时间间隔。
A. 周转时间 B. 响应时间 C. 等待时间 D. 运行时间

19. 作业从进入后备队列到被调度程序选中的时间间隔称为 C。
A. 周转时间 B. 响应时间 C. 等待时间 D. 触发时间

20. 假设下述四个作业同时到达，当使用最高优先数优先调度算法时，作业的平均周转时间为 D 小时。

作业	所需运行时间	优先数
1	2	4
2	5	9
3	8	1
4	3	8

- A. 4.5 B. 10.5 C. 4.75 D. 10.25
最高优先数优先。作业执行次序 2 (5)，4 (3)，1 (2)，3 (8)。
周转时间依次是 5 8 10 18, Avg=10.25

21. 设有4个作业同时到达，每个作业的执行时间均为2小时，它们在一台处理机上按单道方式运行，则平均周转时间为 B。

- A. 1 小时 B. 5 小时 C. 2.5 小时 D. 8 小时
作业执行次序 * (2)，* (2)，* (2)，* (2)；周转时间依次是 2 4 6 8。Avg=5

22. 现有3个同时到达的作业J1、J2和J3，它们的执行时间分别是T1、T2和T3，且T1 < T2 < T3。系统按单道方式运行且采用短作业优先算法，则平均周转时间是 C。

- A. T1 + T2 + T3 B. (T1 + T2 + T3) / 3
C. (3T1 + 2T2 + T3) / 3 D. (T1 + 2T2 + 3T3) / 3

SJF顺序 J1, J2, J3。T1, T1+T2, T1+T2+T3。总和 3T1+2T2+T3。平均除以3。

23. 作业周转时间为 B。

- A. 作业开始时间 - 作业提交时间
B. 作业等待时间 + 作业执行时间
C. 作业等待时间
D. 作业执行时间

二、填空题

1. 程序顺序执行时有顺序性、封闭性和可再现性的特点。
2. 进程是一个动态概念，而程序是一个静态概念。
3. 在一个单处理机系统中，若有5个进程，则处于就绪状态的用户进程最多有4个，最少有0个。
4. 一个就绪进程都没有的状态叫做CPU空转。此时刻，现运行进程是0#进程。
5. 进程的调度方式有两种，一种是可剥夺的，另一种是不可剥夺的。
6. 一种最常用的进程调度算法是把处理机分配给具有最高优先权的进程。而确定优先权的方法概括起来不外乎是基于静态特性和动态特性两种方法。前者所得到的是静态优先权，后者所得到的是动态优先权。
7. 设有一组作业，它们的提交时间及运行时间如下：

作业号 提交时间 运行时间（分钟）

1	9:00	70
2	9:40	30
3	9:50	10
4	10:10	5

在单道方式下，采用短作业优先调度算法，作业的执行顺序是1 4 3 2。

8、设有3个作业，A，B，C。其运行时间分别为1，2，3小时。假定这些作业同时到达，并在一台处理器上单道运行，则平均周转时间最小的执行序列是ABC。

9、导致CPU切换现运行进程的事件包括I/O请求（现运行进程入睡）、现运行进程时间片用完、进程结束、更高优先级进程到达或被唤醒。

10、多道系统，资源分配的单位是进程。（可选项：进程，作业，线程）

四、综合题

（一）情景分析，基于多级反馈队列的调度系统。

1、上班时间，n个IO密集型的交互式作业和1个CPU密集型的长作业同时进入系统。分析所有作业的运行情况。

初始状态：所有作业（n+1个）刚进入系统，都放在最高优先级队列（Q8）

I/O 作业的运行情况：

- 由于是交互式作业，它们运行很短时间就会请求 I/O（如等待用户按键）并进入睡眠状态。

- 它们不会用完 Q8 的时间片，因此在唤醒后依然会回到 Q8（或保持在高优先级队列）。
- 结果：它们始终占据高优先级，系统对用户的输入响应非常迅速，交互性好。

CPU 作业的运行情况：

- 该作业需要长时间计算。它在 Q8 运行完一个时间片后，会被降级到 Q7。
- 接着在 Q7 用完时间片，降级到 Q6，以此类推，最终沉底到最低优先级队列（Q1）。
- 结果：该作业主要在 Q1 运行。只有当所有 I/O 作业都处于睡眠状态（即没有高优先级任务）时，它才能获得 CPU。它利用 Q1 的长轮转时间片进行后台计算。

整体运行情况：

- 高层队列里是 n 个 I/O 作业，它们频繁抢占 CPU，但每次占用时间很短。
- 底层队列里是那个 CPU 长作业。只有当高层队列的 I/O 作业都处于等待状态（所有人都空闲）时，CPU 才会分给底层的长作业。
- 这样既保证了交互式用户的响应速度，又利用空闲时间推进了后台长作业。

2、凌晨3点（没有人上机与系统交互了）。。。系统中的作业和它们的运行情况。

I/O 作业的状态：

- 由于没有用户输入（键盘中断），这 n 个交互式作业会一直处于阻塞（睡眠）状态，等待 I/O 完成。
- 它们离开多级反馈队列，不占用就绪队列的资源。

CPU 作业的状态：

- 它是系统中唯一处于“就绪”或“运行”状态的进程。
- 它会独占 CPU。虽然它位于低优先级队列（Q1），但由于高优先级队列为空，调度器会一直选择它运行。

整体运行情况：

- 系统会定期将该 CPU 作业移动到 Q8。它会迅速消耗掉高层队列的短时间片，再次逐级下降（Q8 -> Q7 -> ... -> Q1），大部分时间依然在 Q1 运行。此时系统变身为一个单道批处理系统，CPU 利用率接近 100%，长作业全速运行。

3、某个进程，从 IO密集阶段进入 CPU密集阶段。。。之后，回到 IO密集阶段。

第一阶段：从 I/O 密集 -> CPU 密集（下沉过程）

- 作为 I/O 密集型进程，它原本处于高优先级队列（如 Q8）。
- 从 I/O 密集 -> CPU 密集：它不再频繁请求 I/O，而是开始长时间计算。它连续用完时间片。调度器发现其行为改变，将其逐级降级。该进程会逐渐下沉，最终停留在最低优先级队列（Q1），被识别为后台批处理任务。

第二阶段：从 CPU 密集 -> I/O 密集（恢复过程）

- 它现在位于 Q1。
- 从 CPU 密集 -> I/O 密集：它开始进行 I/O 操作。I/O 完成被唤醒。根据规则，被唤醒的进程，回原先就绪队列。系统周期性地将所有进程（包括这个被困在 Q1 的进程）移动至 Q8。回到 Q8 后，由于它现在是 I/O 密集型的，它会在时间片用完前就请求 I/O 并让出 CPU。它将保持在 Q8（或高优先级区域），重新获得高响应速度。

4、第二天早上，上班时间到。

- 原本阻塞的 n 个 I/O 作业被中断处理程序唤醒。根据规则，它们回到原先的高优先级队列。
- 此时 CPU 可能正在运行那个 Q1 的长作业。由于 Q8 出现了就绪进程，系统会立即剥夺当前 Q1 进程的 CPU，转而调度 Q8 中的交互式作业。
- 系统恢复交互性，用户感觉不到卡顿，CPU 长作业再次退居后台运行。

（二）参考OSTEP 第8章。一个CPU bound，可以通过巧妙的方式垄断CPU。怎么做？系统怎样防作弊。

首先明确多级反馈队列的规则：

- 如果一个进程用完了分配给它的整个时间片，降低其优先级（移入下一级队列）。
- 如果一个进程在时间片用完之前主动放弃CPU（如进行 I/O 操作），保持优先级不变。当它被唤醒后，回到原先就绪队列，重置时间片。

所以，一个进程想要垄断CPU，只需要避免“用完时间片”即可。

具体做法：在代码中插入逻辑，让程序运行一段时间（例如占时间片的 99%），然后发出一个不需要等待很久的 I/O 请求。通过不断重复“运行 99% -> 放弃 -> 恢复高优先级”的过程，该 CPU 密集型进程可以一直霸占高优先级队列的 CPU 配额，导致真正的 I/O 密集型进程无法及时响应，且低优先级的其他进程完全饿死。

系统怎样防作弊呢？

也很简单。既然上述作弊过程是利用了“重置时间片”的漏洞，那么为了防止上述作弊行为，只需要修改时间片的计量规则。

调度器不应该在进程放弃 CPU 时重置其时间片额度，而应该累计该进程在当前优先级总共消耗的 CPU 时间。· 无论进程是连续运行完时间片，还是分多次运行（每次运行一小会儿就放弃），只要它在当前优先级队列中累计使用的 CPU 时间达到了该队列规定的时间片长度，它就会被强制降低优先级，移入下一级队列。

在该策略下，作弊的进程即使频繁让出 CPU，其消耗的时间配额也会不断累积。一旦配额用尽，它依然会被降级，从而无法长期垄断高优先级资源。